Contents

The Theory of Data

Particles, Atoms, Anchors, Universes, and Lawful Transformation

Huayin Wang · datumwise, an independent open-source research project
Reconstructed foundations draft, revised particle ontology · 2026-07-30
Companion system: Columna

Abstract

This paper proposes a foundations theory for analytical data.

Its starting point is simple: a value becomes data when it is typed and located. The location is not an unspecified database key. It is an **anchor point** in a typed coordinate structure. A

particle—also called a key-value pair—is one typed value at one anchor point. An **atom**—also called a column—is a homogeneous, functionally consistent binding of particles over an anchor inside a universe.

The coordinate structure is built from dimensions, axes, coordinates, anchors, and anchor points. A **dimension** is a typed set such as `Day`, `Store`, or `Product`. An **axis** is a hierarchy of compatible dimensions connected by declared coarsening maps, such as `Hour → Day → Month → Year → AllTime`. A **coordinate** is one member of one dimension. An **anchor** selects one level from every axis in a universe. An **anchor point** is one tuple in the resulting product. Every axis terminates in a singleton level, so an axis omitted from an abbreviated anchor has not disappeared: it has been contracted to its terminal point.

A particle has local typed meaning:

$$p = (a, x), \qquad a \in \mathrm{Pts}(A), \quad x \in |X|.$$

Its key is the anchor point a. An atom binds particles that share one value type, one anchor, one universe, and one functional rule:

$$v : S_v \to X, \qquad \mathrm{graph}(v) = \{(a, v(a)) \mid a \in S_v\}.$$

The particle is analytically simple; the atom has organized multiplicity. Variation, hierarchy movement, missingness, aggregation fibers, and blocked reduction become defined at atom level.

A **universe** is not the Cartesian product of dimensions. It is a typed population inhabiting that coordinate structure, together with a law of existence. In an **event universe**, occurrences generate points. In a **spine universe**, membership is established independently of whether a value is present. This distinction determines the operational meaning of absence and gives the missingness anchor a formal jurisdiction.

A governed atom carries three anchors. Its **V-anchor** states where values live. Its **M-anchor** states where absence speaks. Its operator-indexed **B-anchors** state where reduction must stop. Familiar concepts such as stock and flow are not primitives: they are domain explanations for patterns expressed more generally as B-anchor rules over typed axes.

At atom level there are two primitive value-bearing operations. A **mapper** lifts a typed value function pointwise while preserving the anchor. A **reducer** combines a lawful anchor coarsening with an aggregator applied to each fiber of the coarsening. More elaborate operations—restriction, reindexing, lag, windows, allocation, alignment, expansion, and crossing—are constructions in the anchor, frame, or universe calculi. They determine which particles meet; mappers and reducers determine what their values become.

The resulting theory defines lawful analytical movement as a conjunction: the anchor map must be well typed and corroborated; the aggregator algebra must support the requested composition; the atom's family must license the operator; no B-anchor may be spent unlawfully; M-anchor obligations must be respected; and the movement must remain within a universe or cross only through a declared passage. A finite declaration language can make this lawfulness decidable relative to its declared premises.

Meaning is therefore neither an arbitrary gloss attached to a table nor a complete ontology of the physical world. It is an executable contract over typed values, anchor points, particles, atoms, populations, and transformations. A generated grammar maps expressions into that contract; adjudication tests the contract against available data; and an honesty layer determines whether the system may serve, disclose, clarify, or refuse.

---

1. The question

The founding question is:

> What must data be for analytical correctness to have laws?

The conventional answer begins with rows and tables. A row groups fields that happen to be recorded together. A table groups rows that happen to share a storage layout. Those forms are useful for bookkeeping and execution, but neither one supplies a natural home for the laws that determine whether a variable can be aggregated, shifted, aligned, crossed, or interpreted under absence.

The theory begins with typed location.

A value without a location is not yet a data observation. A location without a typed coordinate system is only an opaque identifier. The coordinate system must therefore be defined before the elementary observation that inhabits it.

Once anchors and anchor points are available, the theory has two data levels.

A **particle**, also called a key-value pair, is one typed value at one anchor point. Its key is the anchor point.

An **atom**, also called a column, is a homogeneous typed function binding many particles over an anchor in a universe.

The particle has local typed meaning:

> value x of type X at anchor point a.

The atom has organized multiplicity. Relations among anchor points make variation, absence, hierarchy movement, aggregation, and transformation law possible.

The central claim is:

> Analytical meaning becomes governable at the atom because the atom binds homogeneous particles over a typed anchor inside a universe.

The particle is not meaning-free. It is analytically simple. The atom is the first object with enough internal structure for stable analytical identity and lawful behavior.

The chemistry vocabulary is teaching scaffolding, not proof. The substance of the theory is the typed coordinate system, the particle as located value, the atom as functionally consistent binding, and the legal contract governing transformation.

---

2. Scope

The theory concerns **analytical data**: typed values distributed over populations and transformed to answer questions across coordinates.

Its strongest scope includes:

- measures and attributes indexed by typed dimensions;
- event and spine populations;
- aggregation and re-aggregation;
- grain change;
- missing-value obligations;
- blocked reductions;
- frame-level derivation;

- alignment and crossing between declared territories;
- certified analytical plans.

The theory does not claim, in its present form, to provide a complete ontology for every possible artifact called data. Text, images, arbitrary graphs, programs, probability models, counterfactual worlds, and unstructured documents may be representable within or above the framework, but they are not required to establish the analytical core.

The scope should therefore be stated honestly:

> This is a working theory of lawful transformation for analytical data, with a program toward a broader theory of data.

---

3. Pure value types

Let X be a **pure value type**.

A value type is not merely a physical storage representation. Two variables may both be stored as decimals yet remain different types. Conversely, one logical type may have several physical encodings.

A type may determine:

- its carrier set $|X|$;
- its computational representation;
- admissible pointwise functions;
- equality and ordering behavior;
- unit and scale, where those are part of the type;
- the output types of typed functions.

Examples might include:

$$\text{RevenueUSD}, \qquad \text{CostUSD}, \qquad \text{TemperatureC}, \qquad \text{ProductSet}.$$

Even when:

$$|\text{RevenueUSD}| = |\text{CostUSD}| = \mathbb{R},$$

the types remain nominally distinct:

$$\text{RevenueUSD} \neq \text{CostUSD}.$$

The type X is the narrow connection between the data realm and whatever originated the values. It can be thought of as a **wormhole**: an opaque type identity and its value algebra enter the data realm, without requiring the analytical kernel to import a full ontology of customers, stores, contracts, sensors, or physical objects.

This distinction is important.

The type X does **not** need to encode whether a value is:

- observed;
- measured;
- inferred;
- forecast;
- planned;

- simulated.

Those are superstructures that use values of type X. They are not constitutive of the pure value type itself.

Likewise, temporal behavior such as ”stock” or ”flow” is not required as a primitive property of X. Operational restrictions of that kind are represented more generally through B-anchors over typed axes.

---

4. Typed dimensions

A **dimension** is a named typed set.

Examples include:

$$\text{Hour}, \quad \text{Day}, \quad \text{Month}, \quad \text{Year},$$

$$\text{Store}, \quad \text{Region},$$

$$\text{Product}, \quad \text{Category}.$$

Each dimension D has a carrier set $|D|$. Its members are typed coordinates:

$$\text{2026-07-30} : \text{Day},$$

$$\text{Store}_{17} : \text{Store},$$

$$\text{SKU}_{42} : \text{Product}.$$

Type identity is not reducible to storage representation. Store identifiers and product identifiers may both be strings, but:

$$\text{Store} \neq \text{Product}.$$

A dimension is therefore not merely a database column or key. It is a coordinate type.

When a dimension participates in a hierarchy, it may also be called a **dimension level**. The term *dimension* names the typed set. The term *level* names its position within an axis.

---

5. Axes

An **axis** is a typed hierarchy of compatible dimension levels connected by declared functional coarsening maps.

A time axis may be:

$$\text{Hour} \to \text{Day} \to \text{Month} \to \text{Year} \to \text{AllTime}.$$

A location axis may be:

$$\text{Store} \to \text{Region} \to \text{AllLocation}.$$

A product axis may be:

$$\text{Product} \to \text{Category} \to \text{AllProduct}.$$

An edge such as:

$$d_m : \text{Day} \to \text{Month}$$

states that every day reaches exactly one month. An edge such as:

$$s_r : \text{Store} \to \text{Region}$$

states that every store reaches exactly one region.

These maps are declarations, not consequences of naming. They may be corroborated against the data at publication time. If a supposed hierarchy edge is not functional in the actual data, it is not a weaker guarantee. It is an unavailable transport.

5.1 Axes are typed

Axis typing makes malformed coordinate movement rejectable before any aggregation is considered.

A map from `Day` to `Month` may be well typed because both are levels of the time axis. A map from `Store` to `Month` is ill typed because the source and target belong to unrelated axes.

The kernel need not hardcode a distinguished time axis. Time, product, location, scenario, organization, version, or any other axis is treated through the same machinery:

- typed levels;
- hierarchy edges;
- terminal contraction;
- M-anchor obligations;
- B-anchor prohibitions.

5.2 Terminal levels

Every axis α has a terminal singleton level:

$$\top_\alpha = \{*_\alpha\}.$$

For the time axis:

$$\text{AllTime} = \{*_{\text{time}}\}.$$

For the product axis:

$$\text{AllProduct} = \{*_{\text{product}}\}.$$

Every dimension D on the axis has a unique map:

$$!_D : D \to \top_\alpha.$$

This terminal level gives exact meaning to the omission of an axis from abbreviated anchor notation.

An omitted axis has not become undefined. It has been contracted to its terminal point.

---

6. Coordinates, anchors, and anchor points

A **coordinate** is one member of one dimension.

For example:

$$m = \text{July 2026} : \text{Month},$$

$$s = \text{Store}_{17} : \text{Store}.$$

A coordinate is one typed component of a multidimensional address.

An **anchor** selects exactly one dimension level from every axis in a universe.

Suppose a universe has the axes:

$$\text{location}, \qquad \text{product}, \qquad \text{time}.$$

An anchor A may select:

$$A(\text{location}) = \text{Store},$$

$$A(\text{product}) = \text{AllProduct},$$

$$A(\text{time}) = \text{Month}.$$

Its abbreviated notation is:

`{store, month}`

The product axis is omitted because it is at `AllProduct`.

Formally, an anchor is a typed level assignment:

$$A : \text{Axes}(U) \to \text{Levels}$$

that selects one level from each axis.

An anchor may not select two levels from the same axis. Thus:

`{store, month}`

is well formed when `Store` and `Month` belong to different axes, while:

`{day, month}`

is not a well-formed anchor if both belong to time.

The **anchor space** is:

$$\mathrm{Pts}(A) = \prod_{\alpha \in \mathrm{Axes}(U)} |A(\alpha)|.$$

Singleton terminal factors may be omitted from display. Therefore:

$$\mathrm{Pts}(\{\mathrm{Store}, \mathrm{Month}\}) \cong |\mathrm{Store}| \times |\mathrm{Month}|.$$

An **anchor point** is one member of this product:

$$a \in \mathrm{Pts}(A).$$

For example:

$$a = (\mathrm{Store}_{17}, \mathrm{July\ 2026})$$

is an anchor point at `{store, month}`.

The vocabulary is therefore:

- **coordinate**: one typed component;
- **anchor**: the multidimensional coordinate schema;
- **anchor point**: one complete coordinate tuple.

---

7. Particles

Let X be a pure value type, let A be an anchor, and let:

$$a \in \mathrm{Pts}(A)$$

be an anchor point.

An **X-typed particle over** A is:

$$p = (a, x), \qquad x \in |X|.$$

The first component is the key:

$$\mathrm{key}(p) = a.$$

The second component is the value:

$$\mathrm{value}(p) = x.$$

The key is therefore not an arbitrary identifier. It is a complete anchor point: one coordinate from every axis selected by the anchor.

When the type or anchor context must be made explicit, write:

$$p = (X, A, a, x).$$

Usually the shorter pair (a, x) is sufficient because X and A are fixed by context.

7.1 Key-value and particle

A **key-value pair** is another name for a particle when:

- the key is interpreted as an anchor point;
- the value has a declared type;
- the pair belongs to the stated anchor context.

Thus:

`key → value`

has the formal reading:

$$a \mapsto x, \qquad a \in \mathrm{Pts}(A), \quad x \in |X|.$$

This definition distinguishes a particle from an untyped storage pair. The same physical string may be a store coordinate, a product coordinate, or an opaque identifier. It becomes a particle key only through its anchor type.

7.2 Local meaning

A particle has local typed meaning:

> This value x of type X occurs at this anchor point a.

A particle does not yet contain relations among several anchor points. It therefore cannot, by itself, exhibit:

- variation over an axis;
- missingness relative to an expected population;
- hierarchy movement;
- aggregation fibers;
- blocked reduction;
- composition stability.

Those properties are not absent because the particle is meaningless. They are absent because the particle is simple.

7.3 Particle identity

Two particles are identical only when their relevant typed components agree.

For example:

$$(\mathrm{RevenueUSD}, a, 100) \neq (\mathrm{CostUSD}, a, 100).$$

Likewise:

$$(X, a, x) \neq (X, b, x)$$

when $a \neq b$.

The physical representation of the value is therefore insufficient to establish particle identity. Type and anchor point are constitutive.

7.4 Particle and function evaluation

A particle may be read as one evaluation of a function:

$$v(a) = x.$$

Equivalently, it is one member of the graph of that function:

$$(a, x) \in \text{graph}(v).$$

This observation does not eliminate the particle. It establishes the precise relation between the particle and the atom: a particle is one located value; an atom is the functional binding of such particles.

---

8. Universes

An anchor space describes all structurally possible points. It does not state which points belong to one data territory or what the absence of a point means.

A **universe** supplies that missing structure.

Write:

$$U = (\kappa, A_U, P_U),$$

where:

- κ is the universe type;
- A_U is the universe's native anchor;
- $P_U \subseteq \text{Pts}(A_U)$ is its population of anchor points.

The anchor supplies possible addresses. The universe supplies inhabited territory and its existence law.

Two universes may share the same anchor while remaining different. Transactions, inventory, actuals, and forecasts may all use `{store, product, day}` but have different populations, support rules, and lawful crossings.

The complete grain is therefore:

$$\text{Grain} = (U, A).$$

An anchor without a universe does not fully identify the population being described.

8.1 Event universes

In an **event universe**:

$$\kappa = \text{event}.$$

Membership is generated by occurrence.

A transaction point exists because a transaction occurred. A shipment point exists because a shipment occurred. A click point exists because a click occurred.

Event universes are naturally sparse. The absence of a point generally means that no event generated that coordinate. It does not automatically mean that a value is missing.

8.2 Spine universes

In a **spine universe**:

$$\kappa = \text{spine}.$$

Membership is established independently of whether a value is present.

A store-product-day point may belong because the spine declares that every active store-product pair is expected on every relevant day. If an atom has no value at such a point, the coordinate still exists.

Spine universes are expectation-generated. Membership precedes value presence.

The difference can be stated compactly:

> Events generate coordinates. Spines await values.

This distinction is prior to statistical accounts of missingness. It determines where absence can first become an operational fact.

8.3 Population at another anchor

A universe may be represented at a coarser anchor through a lawful anchor map.

If:

$$q : A_U \to A,$$

then the population at anchor A is obtained from the image of the native population, subject to the universe's rules:

$$P_{U,A} = q(P_U).$$

The universe is therefore not tied to one physical table or one stored grain. It is a coordinate territory capable of lawful representation at several anchors.

---

9. The atom

Let X be a pure value type, U a universe, and A an anchor in that universe.

An **atom**, also called a column, is a homogeneous typed partial function:

$$v : P_{U,A} \rightharpoonup X.$$

Equivalently:

$$v : S_v \to X, \qquad S_v \subseteq P_{U,A}.$$

Here:

- $P_{U,A}$ is the universe population at anchor A;
- S_v is the support on which the atom currently has values;
- every value has the same type X;
- every key is an anchor point in the same anchor space.

The graph of the atom is:

$$\text{graph}(v) = \{(a, v(a)) \mid a \in S_v\}.$$

Every member of this graph is a particle:

$$p_a = (a, v(a)).$$

The atom can therefore be described in two equivalent ways:

1. as the typed partial function v;
2. as the functionally consistent binding of its particles.

The functional form is primary for calculation. The particle-family form makes the ontology explicit.

9.1 Column and atom

A **column** is another name for an atom when the column is understood semantically rather than only as a physical vector.

A physical vector becomes an atom only when it has:

- a declared value type;
- an anchor;
- anchor-point keys, explicit or recoverable;
- a universe;
- functional consistency;
- a governed transformation contract.

The correspondences are:

Theory	Conventional form
anchor point	key
particle	key-value pair

Theory	Conventional form
atom	column
atom support	populated column keys
atom graph	the column's key-value pairs

9.2 Functional consistency

A binding of particles is an atom only when it defines a function. For a fixed atom, one anchor point may not carry two unresolved values:

$$(a, x_1), (a, x_2) \in \text{graph}(v) \quad \Longrightarrow \quad x_1 = x_2.$$

If a physical table contains several rows with the same apparent key, then at least one of the following is true:

- an event-identity coordinate is missing from the anchor;
- the rows represent revisions or alternatives that require another axis;
- a prior reduction is required;
- the data has not yet been resolved into an atom.

Multiplicity is not erased. It must be represented in the anchor or resolved by a declared operation.

9.3 The atom as the first lawful unit

A particle has a type, anchor point, and value. It is locally meaningful.

But one particle has no internal coordinate relations. It cannot, by itself, exhibit:

- variation across points;
- missingness relative to expected support;
- hierarchy behavior;
- aggregation fibers;
- blocked reductions;
- stable identity through transformations over a population.

Those properties become defined for the bound function.

The atom is therefore the smallest object with sufficient internal structure for analytical law.

9.4 Particle restriction and recovery

Every particle of an atom can be recovered by evaluation:

$$p_a = (a, v(a)).$$

Conversely, a functionally consistent homogeneous family of particles determines an atom:

$$\{(a, x_a) \mid a \in S\} \quad \Longleftrightarrow \quad v : S \to X, \qquad v(a) = x_a.$$

Thus particle and atom are not competing foundations. They are adjacent levels:

> A particle is one typed value at one anchor point. An atom is the lawful functional binding of particles over an anchor.

---

10. The three anchors of a governed atom

The value function is the atom's mathematical core. A governed atom also carries a legal contract:

$$C = (X, U, A_V, v, A_M, B, F),$$

where:

- X is the pure value type;
- U is the universe;
- A_V is the V-anchor;
- v is the partial value function;
- A_M is the M-anchor;
- B is the operator-indexed family of B-anchors;
- F is the declared operator or measure family.

The three anchors have one coordinate language but different jurisdictions.

10.1 V-anchor: where values live

The **V-anchor** is the atom's value domain:

$$A_V = A.$$

It states the typed coordinates over which the atom varies.

The complete grain is:

$$(U, A_V).$$

10.2 M-anchor: where absence speaks

The **M-anchor** states where the absence of a value has operational significance.

The first interpretation of absence comes from universe type:

- in an event universe, a point may not exist because no event occurred;
- in a spine universe, a point may exist even when the atom has no value.

The M-anchor refines that default. It identifies the axes or anchor structure over which expected presence, leakage, coverage, or disclosure obligations apply.

A useful reading is:

> The V-anchor states where a value may live. The M-anchor states where a missing value can make a claim.

For a spine atom, the relevant missing set may be:

$$P_{U,A_M} \setminus S_v.$$

For an event atom, coordinate absence is not ordinarily a leak. Missingness may instead arise when:

- an event point exists but one typed value is absent;
- the event universe is aligned to a spine that establishes expected coordinates;
- a crossing or restriction changes the served population.

The M-anchor should not be reduced to one statistical category such as MAR or MNAR. Its foundational role is structural: it defines the coordinate jurisdiction of absence. Statistical or causal explanations may be declared above it.

10.3 B-anchor: where motion stops

A **B-anchor** states which coordinate structure an operation may not spend.

B-anchors are indexed by operator or operator family:

$$A_B^g \subseteq A_V.$$

For an aggregator g, a reduction is unlawful when its anchor map coarsens or contracts a blocked component.

This generalizes familiar domain concepts without hardcoding them.

For example, the explanation:

> Inventory is a stock with respect to time.

can be compiled into the operational rule:

> `sum` may not contract or coarsen the atom's selected time level.

The kernel does not need the word *stock*. It needs the typed axis and the blocked movement.

The same machinery can block reduction along:

- product;
- location;
- scenario;
- version;
- organization;
- any declared axis.

Thus the three-anchor summary is:

> **V says where values live. M says where absence speaks. B says where motion stops.**

---

11. Families and operator law

Aggregator algebra states what an operator can do in general. It cannot state what that operator may do to a particular atom.

The atom therefore declares an operator or **measure family** F.

A family specifies, at minimum:

- which aggregators are admitted;
- the input and output value types;
- which axis movements are licensed;

- any required sufficient statistics;
- the relevant B-anchors;
- conservation or disclosure obligations.

The distinction between algebra and family is essential.

An aggregator may be algebraically well behaved yet semantically prohibited for an atom. Summation is associative and commutative, but that does not license summing every numeric atom across every axis.

Lawfulness requires both:

$$\text{AlgebraPermits}(g)$$

and:

$$\text{FamilyPermits}(C, g, q).$$

The algebra permits. The atom's family licenses or refuses.

---

12. Frames, rows, and tables

A **frame** is a family of atoms placed over a common anchor and universe, with a declared support policy.

If:

$$v_1 : A \to X_1, \qquad v_2 : A \to X_2,$$

then a frame may be written:

$$\langle v_1, v_2 \rangle : A \to X_1 \times X_2.$$

A frame can carry genuinely joint meaning. Paired measurements, numerator-denominator structures, intervals, coordinates, and correlated values may require frame-level constraints.

This qualification matters: the theory does not claim that all meaning is reducible to mutually independent columns. It claims that the physical row and table do not automatically establish a semantic unit.

A **row** is a physical or logical co-recording of several particles whose keys are coincident or related. A **table** is a recording and execution convention. Either may encode particles, atoms, or frames, but neither receives semantic authority merely from its container form.

The correct distinction is:

> Semantic structure may live at atom, frame, or universe level. Row and table are not semantic units by default.

Columnar formats converged on vectors for performance reasons. The theory supplies the missing coordinate and legal structure that turns those vectors into governed atoms.

---

13. The two primitive atom-level operations

The statics state what atoms are. The dynamics state how their values may change.

At atom level there are two primitive **value-bearing** operations:

1. mapper;
2. reducer.

This does not mean that every structural operation on data is one of the two. Restriction, reindexing, expansion, alignment, and crossing operate on anchors, frames, or universes. Mapper and reducer are the two primitive ways that values themselves participate in a lawful atom transformation.

13.1 Mapper

Let:

$$f : X \to Y$$

be a typed value function.

For an atom:

$$v : S \to X,$$

the mapper is:

$$\mathrm{map}(f)(v) = f \circ v,$$

so:

$$\mathrm{map}(f)(v) : S \to Y.$$

For every anchor point a:

$$(\mathrm{map}(f)(v))(a) = f(v(a)).$$

A mapper preserves the anchor and support. It changes values pointwise.

The defining locality property is:

> One output value consults one input value at the same anchor point.

Frame-level derivation generalizes the same rule. Given co-anchored atoms:

$$v_i : A \to X_i,$$

and a typed function:

$$f : X_1 \times \cdots \times X_n \to Y,$$

the frame mapper produces:

$$a \mapsto f(v_1(a), \dots, v_n(a)).$$

Revenue minus cost, numerator divided by denominator, and similar derivations are frame-level pointwise mappers.

13.2 Reducer

Let:

$$q : A \to B$$

be a lawful anchor coarsening, and let:

$$g : \mathrm{Bag}(X) \to Y$$

be an aggregator.

For every output anchor point:

$$b \in \mathrm{Pts}(B),$$

the anchor map determines a fiber:

$$q^{-1}(b) = \{a \in \mathrm{Pts}(A) \mid q(a) = b\}.$$

The reducer is:

$$(q^g_* v)(b) = g\left(\{v(a) \mid q(a) = b\}\right).$$

The reducer therefore contains two irreducible parts:

1. a typed anchor map that determines which particles enter each fiber;
2. an aggregator that determines what their values become.

The mapper acts on the value side of each particle while preserving its anchor point. The reducer pushes the atom forward along a coarsening of its coordinate side.

---

14. Coarsening, contraction, and aggregation fibers

Anchors are ordered by lawful coarsening.

Write:

$$A \succeq B$$

when every level selected by A can be mapped lawfully to the corresponding level selected by B.

For example:

$$\{\text{Store}, \text{Day}\} \succeq \{\text{Store}, \text{Month}\} \succeq \{\text{Store}\}.$$

The first movement maps:

$$\text{Day} \to \text{Month}.$$

The second maps:

$$\text{Month} \to \text{AllTime}.$$

Several terms should be distinguished.

A **reduction axis** is an axis whose selected level changes between input and output anchors.

A **coarsened dimension** maps to another nonterminal level on the same axis:

$$\text{Day} \to \text{Month}.$$

A **contracted dimension** maps to the terminal singleton level:

$$\text{Month} \to \text{AllTime}.$$

Both are reductions. Contraction is the special case in which the axis may disappear from abbreviated output notation.

14.1 Two-anchor notation

The expression:

```
sum(revenue @ {store, month}) AT {store}
```

exposes the reducer's full type.

It states:

- the input atom is used at `{store, month}`;
- the output is requested at `{store}`;
- the time axis is contracted from `Month` to `AllTime`;
- sum is applied to each resulting fiber.

Formally:

$$q_*^{\text{sum}}(\text{revenue}), \qquad q : \{\text{Store}, \text{Month}\} \to \{\text{Store}\}.$$

Expanded:

$$q = \text{id}_{\text{Store}} \times !_{\text{Month}}.$$

For each store s:

$$q^{-1}(s) = \{(s, m) \mid m \in \text{Month}\},$$

and:

$$r'(s) = \sum_{m} r(s, m),$$

provided every law with jurisdiction licenses the movement.

The input anchor cannot be omitted safely. The request:

```
sum(revenue) AT {store}
```

does not state whether the source was transaction-, day-, month-, product-month-, or already reduced data. The same output anchor can conceal different lineages and different answers.

The two anchors are therefore part of the reducer's type, not optional metadata.

---

15. Aggregator algebra

Reducers depend on the algebra of their aggregators.

Several properties carry most of the practical weight.

15.1 Monoidality

An aggregator is freely composable under partitioned reduction when its sufficient representation forms a commutative monoid:

- an associative operation;
- a commutative operation, when order is irrelevant;
- an identity.

Sum, count, minimum, and maximum have familiar monoidal forms, subject to their value domains and empty-input conventions.

For a monoidal aggregator, partial reductions can be combined exactly when the fibers form a valid partition and the atom's missingness and multiplicity obligations are respected.

15.2 Decomposability

Some aggregators are not monoids over their displayed output but factor through monoidal sufficient statistics.

Mean is the canonical example. It is not lawful to re-aggregate displayed means without weights. But mean factors exactly through:

$$(\text{sum}, \text{count}),$$

followed by division.

A weighted mean of subgroup means can be exact when subgroup counts or weights are carried. An unweighted mean of means is generally a different statistic.

Exact count-distinct factors through set union, but its exact sufficient state may grow with the number of distinct values. Fixed-size sketches supply approximate summaries and must carry approximation guarantees.

The relevant distinction is not simply ”decomposable or not.” It is:

- exact scalar state;
- exact fixed-size structured state;
- exact unbounded state;
- approximate bounded state.

15.3 Composition stability

Given:

$$A \to B \to C,$$

composition stability asks whether:

$$\text{reduce}(g, A \to C) = \text{reduce}(g, B \to C) \circ \text{reduce}(g, A \to B).$$

For a lawful monoidal aggregation over valid partitions, the equality may hold.

For a displayed mean, it does not generally hold:

$$\text{mean}(\text{means}) \neq \text{mean}(\text{original values}).$$

This is not a warning attached to a query. It is an algebraic fact.

15.4 Algebra is necessary, not sufficient

Even perfect composition algebra does not make an operation lawful for every atom.

A sum of sums may be algebraically stable and still violate an atom's B-anchor. Algebra states what can compose. The atom's family and anchors state what may compose.

---

16. The anchor calculus

Many data operations do not directly transform values. They construct or alter the coordinate structure over which value operations act.

These belong to an **anchor calculus**.

16.1 Restriction

Given a coordinate predicate:

$$P : A \to \{\text{true}, \text{false}\},$$

restriction forms:

$$A_P = \{a \in A \mid P(a)\}$$

and restricts the atom:

$$v|_{A_P}.$$

No value transformation occurs.

A value-dependent filter can be decomposed into:

1. a mapper producing a Boolean condition;
2. an anchor restriction using that condition.

The theory should distinguish ordinary restriction from **carving**, where a predicate defines a named population or subuniverse.

16.2 Reindexing

A genuine reindexing is an anchor isomorphism:

$$r : A \cong B.$$

It renames or reorganizes coordinates without changing values:

$$v'(b) = v(r^{-1}(b)).$$

If the map is many-to-one, it is not mere reindexing; it is coarsening and requires a reducer. If it is one-to-many, it is expansion and requires a value distribution law.

16.3 Expansion

An expansion relates one source point to several target points:

$$e : A \rightrightarrows B.$$

The relation alone does not determine what happens to a value. It may be:

- copied;
- divided;
- weighted;
- assigned to one target;
- refused.

Expansion is therefore an anchor construction, not a third primitive atom-level value operation.

16.4 Neighborhood construction

Window operations begin by constructing a neighborhood relation:

$$N \subseteq A \times A,$$

where $b \in N(a)$ means that source point b contributes to the window at target point a.

The neighborhood belongs to the anchor calculus. The window aggregator remains a reducer.

---

17. Derived operations

Many familiar analytical operations can be expressed as compositions of anchor constructions, mappers, and reducers.

17.1 Lag and lead

Let:

$$\text{pred} : A \rightharpoonup A$$

be a typed predecessor map.

Lag is a pullback along that anchor map:

$$v_{\text{lag}}(a) = v(\text{pred}(a)).$$

A period-over-period difference then consists of:

1. anchor shift;
2. alignment of current and lagged atoms;
3. pointwise frame mapper.

No third atom primitive is required.

17.2 Window functions

Given a neighborhood $N(a)$, a window aggregate is:

$$w(a) = g\{v(b) \mid b \in N(a)\}.$$

It can be decomposed into:

1. neighborhood relation;
2. expansion of each source point to the windows containing it;
3. reduction by target point.

Schematically:

$$\text{window}_{N,g} = \text{reduce}_g \circ \text{expand}_N .$$

The hard semantic questions concern the anchor relation:

- which axis is ordered;
- whether bounds are coordinate-based or row-based;
- how gaps behave;
- whether the current point is included;
- what happens at boundaries.

The value operation is still aggregation over a fiber.

17.3 Allocation

Let:

$$R \subseteq A \times B$$

be a source-to-target relation, and let:

$$w : R \to \mathbb{R}$$

provide weights.

A conservative allocation requires:

$$\sum_{b:(a,b)\in R} w(a,b) = 1$$

for every source point a.

The operation decomposes as:

1. expand source points along R;
2. map each carried value to $w(a,b)v(a)$;
3. reduce by target point.

Thus:

$$\boxed{\text{expand} \to \text{map} \to \text{reduce}}$$

is a normal form for allocation.

Conservation follows when the weight and summation laws hold:

$$\sum_b v'(b) = \sum_a v(a).$$

Attribution may use a similar structure without conservation. Allocation and attribution should therefore remain distinct declarations.

17.4 Broadcast

Broadcast is expansion plus a declared copy law.

Copying one value to many target points is not intrinsically neutral. It may be lawful for labels, parameters, invariant attributes, or membership facts. It may be unlawful for additive measures.

17.5 Explode and unnest

For a set-valued type:

$$v(a) \in \text{Set}(Y),$$

explode introduces an element axis and produces one Y-typed value per member.

It changes both value type and anchor. It can be represented through:

1. a type-supported decomposition;
2. anchor expansion;
3. formation of a new atom.

It is a derived structural operation, not a primitive analytical law.

---

18. Alignment, joins, unions, and physical operations

18.1 Join is not a semantic primitive

A database join bundles several distinct decisions:

- coordinate matching;
- support selection;
- universe crossing;
- multiplicity propagation;
- unmatched-point policy;
- value duplication;
- frame construction.

Its behavior depends on physical choices such as inner, left, and outer joins. The word *join* therefore does not identify one analytical operation.

A join may implement a lawful plan, but the theory should describe that plan in terms of:

$$\text{anchor relation} + \text{support rule} + \text{universe passage} + \text{frame construction}.$$

The physical operator receives no intrinsic semantic authority.

18.2 Alignment

Alignment has a narrower and useful theoretical meaning:

> Alignment constructs a frame from atoms placed over a common typed anchor without changing their values.

If two atoms already share anchor, universe, and compatible support:

$$v_1 : A \to X_1, \qquad v_2 : A \to X_2,$$

alignment forms:

$$\langle v_1, v_2 \rangle : A \to X_1 \times X_2.$$

When supports differ, inner, left, and outer policies are not neutral mechanical details. Universe type and M-anchor must determine whether a canonical common support exists. Otherwise the operation requires disclosure, clarification, or refusal.

Alignment is a frame constructor, not an atom-level primitive.

18.3 Union and concatenation

Physical union or concatenation has no single intrinsic law.

When two atom fragments share:

- value type;
- universe;
- V-anchor;
- M-anchor;
- B-anchors;
- family;

- disjoint supports,

their graphs may be merged:

$$v = v_1 \cup v_2.$$

If supports overlap, a conflict policy is required. If universes or contracts differ, the operation may instead be a universe union, crossing, frame construction, reconciliation, or refusal.

The physical word *union* does not decide.

---

19. The universe calculus

Operations within one universe are not sufficient for all analytical questions. Warehouses contain several data territories.

Between universes lie boundary operations.

Two broad passages are especially important:

1. **alignment at a shared grain**, when the universe laws support a common anchor and support interpretation;
2. **crossing through a declared face**, when a many-to-many or otherwise nonfunctional frontier requires an explicit assignment, allocation, touch, or other passage rule.

A general relation between universes does not, by itself, define value movement. It supplies possible passages. A declared face supplies the law.

Examples include:

- deliberate overcounting through a `touch` face;
- single assignment through a `primary` face;
- conserved allocation through a weighted `split` face.

Each face has its own consequences, conservation rules, discarded memberships, and disclosures.

There is no lawful third path in which a many-to-many relationship silently duplicates values and presents the result as self-explanatory.

---

20. A normal form for analytical transformation

The combined calculus suggests a stronger research claim:

> Every lawful analytical transformation in the supported grammar can be expressed as a composition of anchor construction, pointwise mapping, and fiber reduction.

Schematically:

$$\boxed{\text{anchor structure} \to \text{map} \to \text{reduce}}$$

Examples include:

- restriction: anchor construction only;
- reindexing: anchor isomorphism;
- derived measure: alignment plus map;
- lag: anchor shift plus alignment plus map;

- window: neighborhood expansion plus reduce;
- allocation: expansion plus weighted map plus reduce;
- dimensional roll-up: anchor coarsening plus reduce;
- broadcast: expansion plus copy map;
- crossing: declared universe relation plus face-specific map/reduce.

This is not yet a proved representation theorem. It is a formal research target.

A completeness theorem would need to state the supported grammar precisely and prove that every admitted plan decomposes into these forms.

---

21. Lawful transformation

A transformation is lawful only when every clause with jurisdiction licenses it.

For a reducer:

$$q_*^g : C \to C',$$

the obligations include at least the following.

21.1 Type obligation

The input and output value types must match the aggregator's signature:

$$g : \mathrm{Bag}(X) \to Y.$$

21.2 Anchor obligation

The map:

$$q : A \to B$$

must be assembled from lawful axis transitions:

- identity;
- corroborated hierarchy coarsening;
- terminal contraction.

21.3 Algebra obligation

The aggregator's algebra must support the planned decomposition, partial aggregation, approximation, and composition.

21.4 Family obligation

The atom's declared family must license g over the axes changed by q.

21.5 B-anchor obligation

The reduction may not spend any blocked anchor component for the chosen operator family.

21.6 M-anchor obligation

The reduction must account for absence over every coordinate domain where the M-anchor has jurisdiction. The result may require disclosure, adjustment, refusal, or a change in the claim being made.

21.7 Universe obligation

The movement must remain within one universe or pass between universes only through a declared alignment or face.

21.8 Population obligation

Restriction, carving, alignment, and crossing must preserve or explicitly state the served population.

The closure can be stated compactly:

$$\text{Lawful}(T, C) \iff \bigwedge_{j \in \text{Jurisdiction}(T,C)} \text{Licensed}_j(T, C).$$

This conjunction is what makes lawfulness mechanically checkable.

---

22. Decidability and premise status

A finite declaration language can make transformation lawfulness decidable **relative to its premises**.

This qualification is essential.

Some declarations can be directly checked:

- type conformance;
- anchor uniqueness;
- hierarchy functionality;
- support coverage;
- weight normalization;
- conservation;
- forbidden-axis movement.

Others may be corroborated only partially:

- completeness of a spine;
- stability of a mapping;
- representativeness of a population.

Still others may be assumptions that the data cannot identify:

- the external correctness of a unit label;
- the intended business population;
- a causal missingness mechanism;
- whether a declared variable name matches the world as intended.

The model should therefore attach an epistemic status to each declaration, such as:

- **verified**;
- **corroborated**;

- **assumed**;
- **unidentifiable from available data**;
- **contradicted**.

The kernel's guarantee is then relative soundness:

> Given the declared contract and the recorded evidential status of its clauses, the kernel determines whether a transformation preserves that contract.

It should not claim that all declarations are objectively true merely because the available values fail to contradict them.

This status system fits naturally with the honesty contract.

---

23. Meaning

Meaning in this theory is operational and compositional.

A governed atom is not just a vector of values. It is:

$$C = (X, U, A_V, v, A_M, B, F, \mathcal{E}),$$

where $\mathcal{E}$ records the evidential status of its declarations.

Its meaning consists in the lawful role of the atom:

- what type of values it contains;
- where those values live;
- which population those points belong to;
- where absence has jurisdiction;
- which movements are prohibited;
- which aggregators and compositions are admitted;
- what may be derived or crossed;
- what must be disclosed.

This is not full referential meaning in the sense of a complete ontology of the physical world. Nor is it merely syntax.

It is **inferential and operational meaning**: the set of lawful transformations and claims admitted by the governed object.

The pure type X is the narrow wormhole to the external world. Once the value enters, the analytical laws are stated in domain-independent terms:

- typed coordinates;
- universes;
- fibers;
- aggregators;
- conservation;
- blocked movement;
- declared passages.

A mean-of-means error has the same form in retail, epidemiology, and astronomy. The law does not need to know what the values refer to.

This supports a refined autonomy thesis:

The data realm is formally and operationally autonomous, though not semantically sealed. Its types enter from outside; its analytical movement is governed by intrinsic coordinate and transformation laws.

---

24. Meaning as a mapping

A declared analytical system supports a mapping:

$$\text{expression} \longrightarrow \text{declared model} \longrightarrow \text{governed data}.$$

The two arrows should be distinguished carefully.

24.1 Expression to model

When a grammar is generated from a declared model, every well-formed expression names objects and operations admitted by that model.

The first arrow can be total by construction:

- every measure name resolves;
- every coordinate belongs to a declared dimension;
- every anchor is well formed;
- every operator has a typed signature;
- every expression has a candidate denotation.

Generated syntax prevents the language from speaking arbitrary container operations that have no place in the model.

24.2 Model to governed data

The second relation is not merely a denotation arrow. It includes satisfaction and evidence:

$$D \models C_j$$

for declarations that can be verified or corroborated against data D.

For some clauses:

$$D \not\models C_j$$

and publication must fail.

For unidentifiable clauses, neither the declaration nor its negation may be derivable from the data. The clause remains an explicit assumption.

The system therefore separates:

1. **denotation**: what an expression means under the model;
2. **typing**: whether the expression is lawful under the model;
3. **satisfaction**: which declarations are supported by the data;
4. **pragmatics**: what the system may responsibly say.

---

25. The honesty contract

The system's answer is a speech act with conditions.

Four response modes capture the principal outcomes.

Serve

The requested expression has one lawful interpretation, all required obligations are discharged, and no material qualification must accompany the result.

Disclose

The expression is lawful, but conditions affecting its claim must travel with the answer.

Examples include:

- coverage shortfall;
- approximation error;
- crossing skew;
- discarded memberships;
- stale support;
- unresolved but nonfatal assumptions.

Clarify

Several lawful interpretations exist, and the system cannot choose among them without importing an undeclared preference.

Clarification is not a failure. It is a successful determination that human choice is part of the meaning.

Refuse

No lawful path exists under the declared model and available evidence.

A refusal is a precision event. It prevents a mechanically executable but semantically undefined result from being served as an answer.

The response mode should be derivable from the same obligations that certify the plan. Disclosure is not prose pasted on after computation. It is a projection of the proof and its undischarged conditions.

---

26. Rows, SQL, and semantic layers

SQL is a powerful grammar of physical containers and execution.

It can express:

- tables;
- rows;
- joins;
- filters;
- groups;
- windows;
- unions;
- projections.

But its syntax does not inherently name:

- input anchors;
- output anchors;
- universe types;
- M-anchors;
- B-anchors;
- crossing faces;
- evidential status.

Those meanings must be supplied externally or inferred from convention.

A conventional semantic layer improves consistency by assigning stable names and definitions to measures and dimensions. That is valuable. But a definition may still be unverified, and a valid metric may still be used at an unlawful lineage, population, or crossing.

The theory's distinctive claim is not that definitions are unnecessary. It is:

> Definitions become trustworthy only when their coordinate and transformation laws are executable and adjudicated.

The row-and-table paradigm did not fail because rows and tables are useless. It failed because it treated storage containers as if they carried the laws of the variables placed inside them.

---

27. Frames and generated analytical plans

A question may request several atoms at a common output anchor.

The resulting plan is naturally a directed acyclic graph:

- carve a universe or population;
- choose an output anchor;
- select governed atoms;
- transport anchors through corroborated hierarchy edges;
- cross universes through declared faces;
- reduce fibers under family law;
- align outputs at a shared anchor;
- derive final values through pointwise frame mappers.

This plan should be represented as an explicit intermediate form rather than hidden inside planner code or SQL generation.

An untrusted searcher—human, static planner, or language model—may propose a plan. A trusted kernel should check two separate obligations:

1. **lawfulness**: the plan violates no law of the model;
2. **faithfulness**: the plan computes the denotation of the ask rather than a different lawful question.

A lawful plan that answers the wrong question is still a silent failure.

The certificate chain is:

$$\text{data} \to \text{adjudicated model} \to \text{certified plan} \to \text{served answer}.$$

---

28. Evidence and theoretical rank

The theory should distinguish several kinds of evidence.

28.1 Internal closure

The vocabulary forms a derivation chain:

typed value → dimension → axis → anchor → universe → atom → V/M/B law → mapper and reducer → fr

Internal closure means that the major concepts are defined in terms of prior concepts and can jointly state lawful movement.

28.2 Compression

Many familiar analytical traps become instances of a smaller vocabulary:

- mean of means: loss of sufficient aggregation state;
- summing inventory across a blocked axis: B-anchor violation;
- missing rows: universe and M-anchor mismatch;
- fan-out: undeclared universe crossing;
- allocation drift: violated face conservation;
- wrong grain: input-anchor substitution;
- ambiguous population: universe or carve ambiguity.

Compression is evidence that the theory has found useful joints.

28.3 Implementation

A running system demonstrates that declarations, adjudication, response modes, and certified plans can be made executable.

Implementation establishes feasibility. It does not by itself establish completeness or broad empirical effectiveness.

28.4 Convergence

Statistics, dimensional modeling, columnar execution, type-level grain work, database aggregation theory, and formal verification all touch nearby structures.

Such convergence is **search evidence**: independent traditions repeatedly find grain, variables, aggregation algebra, and typed transformation important.

It is not confirmation of every ontological claim in the theory.

28.5 Execution

Executed attacks and benchmarks test whether the theory predicts and prevents failures.

The proper maxim is:

> Agreement is search evidence; execution is evidence.

---

29. Falsifiable research program

A mature theory needs claims that can fail.

The following propositions define a stronger research program.

29.1 Soundness

Every plan accepted by the kernel preserves every applicable clause of the declared contract.

29.2 Failure coverage

Every failure within a stated analytical class violates at least one named type, anchor, universe, algebra, family, M-anchor, B-anchor, crossing, or faithfulness obligation.

29.3 Declaration minimality

Removing any declaration category creates a distinguishable class of accepted silent failures.

29.4 Primitive completeness

Every lawful transformation in the supported grammar decomposes into the stated anchor, mapper, reducer, frame, and universe constructions.

29.5 Reproducibility

Independent modelers assign compatible universes, anchors, families, M-anchors, and B-anchors at an acceptable rate.

29.6 Predictive intervention

Adding the theory's declarations prevents held-out failures more effectively than conventional metadata, SQL tests, documentation, or grain-only typing.

29.7 Domain transfer

Contracts and checks developed in one domain detect structurally analogous failures in previously unseen domains.

29.8 Refusal correctness

When the kernel refuses an ask as undefined, no equivalent lawful plan exists under the same declared premises.

29.9 Faithfulness

A certified plan cannot substitute a semantically adjacent but different lawful computation for the ask.

29.10 Adversarial resistance

An adversary cannot construct an accepted but semantically invalid plan without falsifying, contradicting, or exploiting an explicitly unverified premise.

These tests are more demanding than finding examples that fit the vocabulary. They expose the theory to disconfirmation.

---

30. Open problems

The rebuilt foundation resolves several ambiguities in the early theory, but substantial work remains.

30.1 Formal universe boundaries

The exact conditions under which a universe is maximal, and how functional reachability determines its territory, require a formal definition.

30.2 Support transport

The behavior of event and spine populations under coarsening, restriction, and alignment requires explicit laws.

30.3 M-anchor algebra

M-anchor has a structural foundation—where absence speaks—but a complete calculus of leak propagation, disclosure, identity insertion, and refusal remains to be developed.

30.4 B-anchor composition

Operator-indexed blocked anchors must compose through derived measures and frames. The output B-anchor of a mapper or reducer should be derivable from its inputs and operation.

30.5 Approximation

Sketches, sampling, numerical error, and approximate query processing need first-class types and disclosure rules.

30.6 Refinement

Coarsening can discard distinctions lawfully. Refinement requires information not present in the coarser atom unless supplied by a declared spine, allocation, interpolation, or external model.

The theory should make unlawful refinement a theorem rather than a planner policy.

30.7 Normal-form completeness

The anchor–map–reduce normal form is promising but unproved.

30.8 Frame constraints

Joint laws that live irreducibly across several atoms need a formal frame contract.

30.9 Epistemic declarations

The boundary among verified, corroborated, assumed, unidentifiable, and contradicted clauses needs an executable evidence language.

30.10 Canonical plan representation

Certified planning requires a closed intermediate representation, canonical serialization, proof obligations, versioning, and dependency-aware recertification.

---

31. Formal vocabulary

Pure value type X A nominal and algebraic type defining admissible values and pointwise functions
Dimension A named typed coordinate set at one level
Dimension level A dimension understood as one position in an axis hierarchy
Axis A hierarchy of compatible dimensions connected by declared coarsening maps
Terminal level The singleton top level of an axis
Coordinate One member of one dimension
Anchor A selection of exactly one dimension level from each universe axis
Anchor space The Cartesian product of an anchor's selected dimension carriers
Anchor point One tuple in an anchor space
Particle One typed value at one anchor point: $p = (a, x)$
Key-value pair Another name for a particle when the key is an anchor point and the value is typed
Universe A typed population of anchor points together with an existence law
Event universe A universe whose points are generated by occurrences
Spine universe A universe whose points are established independently of value presence
Grain The pair (U, A): an anchor situated in a universe
Support The anchor points at which an atom currently contains particles
Atom A homogeneous typed partial function, equivalently a functionally consistent binding of particles
V-anchor The anchor over which an atom's values live
M-anchor The coordinate jurisdiction in which absence has operational significance
B-anchor An operator-indexed anchor component that a transformation may not spend
Family The atom's declared operator and aggregation law
Anchor map A typed map from one anchor to another
Reduction axis An axis whose selected level changes in a reduction
Coarsened dimension A dimension mapped to a nonterminal coarser level
Contracted dimension A dimension mapped to its axis's terminal singleton level
Aggregation fiber The source anchor points mapping to one target anchor point
Mapper A pointwise typed value function lifted over an atom or frame
Reducer An aggregator applied to the fibers of a lawful anchor coarsening
Restriction Selection of a subspace of anchor points
Reindexing An isomorphism of anchors
Expansion A one-to-many anchor relation requiring a declared value law
Alignment Frame construction from atoms at a common anchor under a lawful support policy
Crossing Movement between universes through a declared face
Face The declared value law governing a universe crossing
Frame A family of co-anchored atoms with joint structure
Lawfulness Satisfaction of every applicable type, anchor, algebra, family, M, B, universe, and population obligation
Faithfulness Agreement between a plan's denotation and the ask's denotation
Certificate A checkable record that a plan discharged its obligations
Honesty contract The rules deriving serve, disclose, clarify, or refuse from adjudication

---

32. Compact formal core

The theory can be summarized by six definitions and two value-bearing operations.

Coordinate system

A universe provides typed axes:

$$\alpha_1, \dots, \alpha_n,$$

each with hierarchy maps and a terminal singleton.

Anchor

An anchor selects one level per axis:

$$A = (D_1, \dots, D_n).$$

Its point space is:

$$\mathrm{Pts}(A) = \prod_i |D_i|.$$

Particle

A particle is one typed value at one anchor point:

$$p = (a, x), \qquad a \in \mathrm{Pts}(A), \quad x \in |X|.$$

Its conventional name is a key-value pair, with:

$$\mathrm{key}(p) = a, \qquad \mathrm{value}(p) = x.$$

Universe

A universe is:

$$U = (\kappa, A_U, P_U).$$

Atom

An atom is:

$$v : P_{U,A} \rightharpoonup X.$$

Equivalently, its particle family is:

$$\mathrm{graph}(v) = \{(a, v(a)) \mid a \in S_v\}.$$

Its conventional name is a column.

Governed atom

A governed atom is:

$$C = (X, U, A_V, v, A_M, B, F, \mathcal{E}).$$

Mapper

$$\text{map}(f)(v) = f \circ v.$$

Reducer

$$(q^g_* v)(b) = g\{v(a) \mid q(a) = b\}.$$

The lawfulness judgment is:

$$\text{Lawful}(T, C) \iff \bigwedge_j \text{Licensed}_j(T, C).$$

The practical reducer notation is:

```
sum(revenue @ {store, month}) AT {store}
```

The input anchor states where the source particles are bound. The output anchor states where the resulting atom will live. The omitted time axis is contracted to `AllTime`. The fiber over each store contains the particles whose month coordinates contract to that store point. Sum acts only if every applicable law licenses that contraction.

---

33. Conclusion

The theory begins neither with rows nor with unlocated values. It begins with typed values placed in typed coordinate space.

A dimension supplies a coordinate type. An axis supplies lawful hierarchy movement. An anchor selects one level from each axis. An anchor point supplies one complete address.

At that point the first data object becomes definable.

A particle is one typed value at one anchor point:

$$p = (a, x).$$

Its key is the anchor point. A key-value pair is therefore another name for a particle.

A universe supplies inhabited territory and the law by which anchor points exist. An atom then binds homogeneous particles over an anchor within that universe:

$$v : S_v \to X.$$

A column is therefore another name for an atom when its keys, type, universe, and functional consistency are understood.

The particle is locally meaningful but analytically simple. The atom has organized multiplicity and is the first object over which variation, missingness, hierarchy movement, aggregation fibers, and transformation law can be defined.

The governed atom gives those laws an explicit home:

- V-anchor: where values live;
- M-anchor: where absence speaks;
- B-anchor: where motion stops;
- family: which operators are licensed.

At atom level, values transform in two primitive ways. Mappers act pointwise on the values of particles while preserving their keys. Reducers use a typed anchor map to form fibers of particles and apply an aggregator to each fiber. Other operations construct the anchor, frame, or universe structures that determine which particles meet.

This division is the center of the theory:

> **Anchor operations determine which particles meet. Mappers and reducers determine what their values may become.**

Rows, tables, joins, unions, and physical plans may encode or implement these laws, but they do not create them.

The result is an executable definition of lawful analytical movement. It is relative to declared premises, honest about what those premises can and cannot prove, and capable of refusing computations that are executable but undefined.

That is the practical promise of a theory of data: not that every meaning can be inferred automatically, but that the meanings on which analytical correctness depends can be given a typed structure, tried against evidence, and made binding on computation.